UNIVERSITÀ DI TRENTO

Introduction to Markup Languages

1

Markup languages

- A markup language is a system for annotating a document in a way that is syntactically distinguishable from the text
- The language specifies a code for formatting, both the layout and style, within a text file. The code used to specify the formatting are called tags
- In most cases is human-readable
- A markup language is NOT a programming language

2

Key terminology

- Tag: a sequence of characters that begins with "<" and ends with ">".

  Three kinds of flavours:

  - Start-tag: e.g. <book>
  - End-tag: e.g. </book>
  - Empty-element tag: e.g.

- Element: a sequence of characters that begins with a start-tags and ends with a matching end-tag, or consists only of a empty-element tag. The characters between the start and the end tag, if any, are the element's content and can contain other elements called children

- Attribute: a name-value pair existing in a start-tag or in a empty element tag

3

Example of Markup Language

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<data>
  <NETWORK>
    <IP>172.150.1.101</IP>
  </NETWORK>
  <LECTURE id="27">
      <COURSE_NAME>Web Programming</COURSE_NAME>
      <LECTURE_NAME>Introduction to XML</LECTURE_NAME>
      <TEACHER_NAME>Giovanna Varni</TEACHER_NAME>
       <TIME>5225.00</TIME>
  </LECTURE>
</data>
```

4

Why markup languages?

Markup:

- is relevant for storing, managing, delivering, structuring, publishing data;
- makes explicit to a machine what is implicit to a person;
- assists us in the re-use of the same information:
  - in different formats
  - in different contexts
  - by different classes of users

5

Types of Markup languages - 1

1) Presentational markup

They are used by traditional word-processing systems: binary codes embedded within document text that produce the WYSIWYG ("what you see is what you get") effect

Such markup is usually hidden from the human users, even authors and editors

Example: the Rich Text Format (RTF) language internally used by Word

6

Types of Markup languages - 2

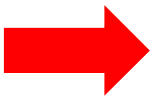
2) Procedural markup

They provide instructions for programs to process the text (add an image and so on), such as e.g. Markdown, and PostScript

The processor runs through the text from beginning to end, following the instructions as encountered

Text with such markup is often edited with the markup visible and directly manipulated by the author

```
/Times-Roman findfont
12 scalefont
setfont
newpath
100 200 moveto
(Example 3) show
```



Example 3

7

Types of Markup languages - 3

3) Descriptive / semantic markup

They are used to label parts of the document for what they are (e.g. define the title of a document), rather than how they should be processed

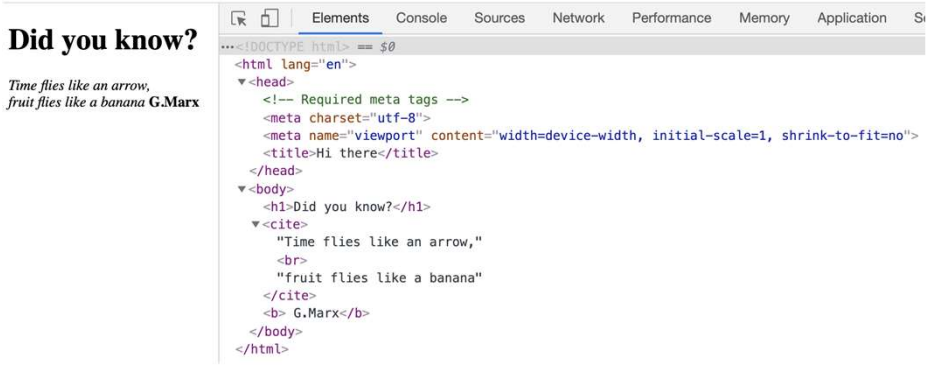
The objective is to decouple the structure of the document from any particular treatment or rendering of it

Descriptive markup encourages authors to write in a way that describes the material conceptually, rather than visually

Examples: LaTeX, HTML, XML

8

Example: HTML



9

The roots: SGML

- SGML is an ISO standard (ISO 8879:1986) which provides a formal notation for the definition of generalized markup languages.
- SGML is not a language in itself. Rather, it is a metalanguage that is used to define other languages.

10

SGML: the three parts

An SGML document is the combination of three parts:

- the content of the document (words, pictures, etc.).This is the part that the author wants to expose to the client;
- the grammar (*DTD – data type definition)* that defines the accepted syntax;
- the stylesheet that establishes how the content that conforms to the grammar has to be rendered on the output device.

Generally, we refer to the parts as files (but they don't have to be separate physical files!).

11

eXtensible Markup Language (XML)

12

What is XML?

XML is a group of technologies created in 1998 from the World Wide Web Consortium for web development

https://www.w3.org/TR/xml/

XML is used (not only) for:

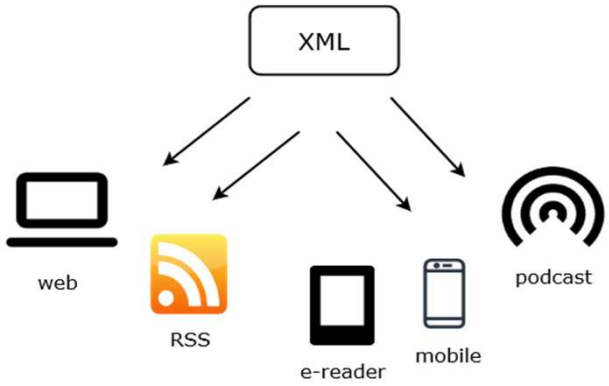
- Having data in a easy format
- Exchanging data
- Accessing to databases
- Changing the format of data
- Writing config files
- Web services

XML does not DO anything! It is just information wrapped in tags

13

Example

The same information defined in XML can be written, modified, displayed by completely different devices for serving completely different purposes



14

Advantages

- Ease:
  - Simple rules
  - Human readable documents
- Compatibility:
  - Platform independent
  - Many tools available for writing and validating docs
- Power:
  - Definition of really complex documents
  - Managed as a text doc when it travels on the web

15

Logical structure of an XML document

- XML declaration (or "Prolog": optional, but if present MUST be the first element):

  <?xml version='1.0' encoding='utf-8'?>
- Optional DTD declaration
- Optional comments and Processing Instructions (PIs)
- The root element's start tag
- All other elements, comments and PIs
- The root element closing tag

16

An XML document

```
<?xml version="1.0" encoding="UTF-8"?>
<note>
  <to>Giovanna</to>
  <from>Myself</from>
  <heading>Reminder</heading>
  <body>Don't forget the Web lecture!</body>
</note>
```

17

XML: Which tags can I use?

- XLM does not use predefined tags, YOU define them!
- Provide a grammar to:
  - define tags
  - define rules for the tags
  - allowed attributes
  - containment rules
- The grammar is defined in a:
  - DTD file
  - XML-Schema file
  - or is NOT DEFINED AT ALL!

18

Attributes vs Elements

- There are no rules about when to use attributes or when to use elements in XML

```
<person gender="female">
  <firstname> Anna </firstname>
  <lastname> Smith</lastname>
</person>
```

```
<person>
  <gender>female</gender>
  <firstname> Anna </firstname>
  <lastname> Smith </lastname>
</person>
```

…both the examples provide the same information

- Some remarks:
  - attributes cannot contain multiple values (elements can)
  - attributes cannot contain tree structures (elements can)

19

Well formed documents

- Tag, element, attribute…
- All XML documents must be well formed, that is they must comply several requirements:
  - XML is case sensitive

    <Pippo> is different from <pippo>
  - there must be a single top-level tag (called the "root tag") that contains all the tags in the document
  - each tag must have a closing tag or, if empty, may provide for the abbreviated form (/>)

    <book></book> or </book>

20

Well formed documents

- All XML documents must be well formed, that is they must comply several requirements:
  - tags must be appropriately nested, i.e., the end tags must follow the reverse order of their respective start tags

    This text is bold and italic NO!

  - attribute values (properties of a tag) must always be enclosed in single or double quotes. You can surround the value with apostrophes (single quotes) if the attribute value contains a double quote. An attribute value that is surrounded by double quotes can contain apostrophes.

    <note date="02/03/2024">
    <to> Pippo </to>
    <from> me </from>
    </note>

21

Well formed documents

- All XML documents must be well formed, that is they must comply several requirements:
  - Empty element can contain attributes

    Note that an empty element can contain one or more attributes inside the start tag:

    <book author= "eckel" price="$39.95" />

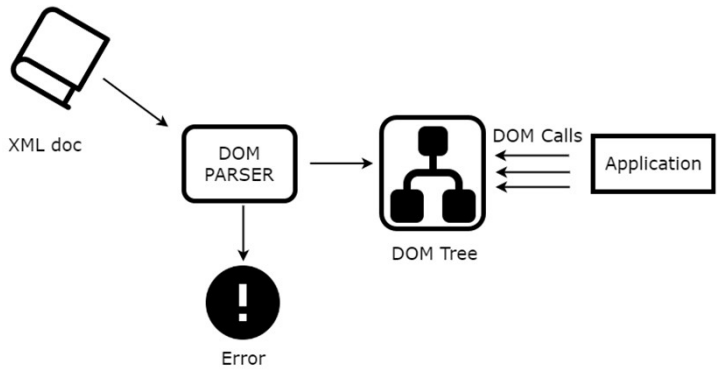
  - No markup characters are allowed

    For a document to be well-formed, it must not have some characters (entities) in the text data e.g.: < >, " ,&.

    If you need for your text to include the < character you can represent it using:
    < or < or &#x3C

22

Checking if a XML doc is well-formed

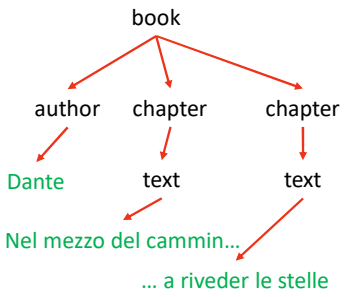


23

XML: tree structure

- An XML document must have a root tag
- An XML document is an information unit that can be seen in two ways:
  - As a linear sequence of characters that contain characters data and markup
  - As an abstract data structure that is a tree of nodes

```
<book>
  <author> Dante </author>
  <chapter id='1'>
     <text> Nel mezzo del cammin…</text>
  </chapter>
  <chapter id='2'>
     <text>… a riveder le stelle</text>
  </chapter>
</book>
```



24

Namespaces

- Since you can define your own tags, if you reuse XML files from other authors you might find tag conflicts
- Conflits can be avoided by declaring a namespace as an attribute of the root element:

```
<table>
  <name> Vangsta </name>
  <width> 80 </width>
  <length>180 </length>
  <height> 120 </height>
</table>

<table>
  <name> Pippo </name>
  <surname> de Pippis </surname>
  <height> 180 </height>
</table>
```

```
<root xmlns:h="https://www.ikea.com/furniture"
xmlns:f="http://www.gym4all.it/tnt/people">

<h:table>
  <h:name> Vangsta </h:name>
  <h:width> 80 </h:width>
  <h:length>180 </h:length>
  <h:height> 120 </h:height>
</h:table>

<f:table>
  <f:name> Pippo </f:name>
  <f:surname> de Pippis </f:surname>
  <f:height> 120 </f:height>
</f:table>

</root>
```

25

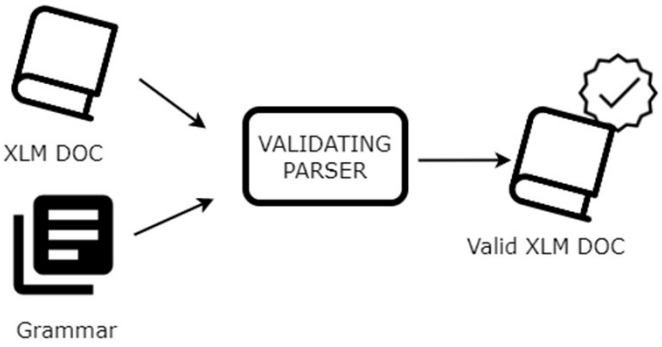
The grammars: DTD and XML - Schema

- A well-formed XML document conforms to the rules (the grammar) of either a DTD or a XML-Schema is a valid XML document
- Validity is not a requirement of XML:
  - An invalid XML document can be a perfectly good and useful XML document
  - A non well-formed document cannot be valid, and is not an XML document
- To make a long story short:

  validation against a DTD or XML – Schema can often be very useful, but is not required

26

Constraining & Validating XML



27

What is a XML Schema?

- DTDs are a little bit complex and they are not written in XML!
- XML Schemas or XML Schemas Definition (XLS) are another W3C standard since 2001
- Each XML document defined through a DTD can be also defined through a XML Schema, the opposite is not necesarily true

28

XML Schema structure

- The < schema > element is the root element of every XML Schema
- <schema> declares the namespace xs (attribute xmlns:xs) whose tag are defined at http://www.w3.org/2001/XMLSchema
- XML Schema elements:
  - Simple elements: they contain only text, not other elements or attributes
  - Complex elements: they contain other element and/or attributes.

    There are four kinds of complex elements:

    - empty elements
    - elements that contain only other elements
    - elements that contain only text (and attributes)
    - elements that contain both other elements and text

29

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="addressbook">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="contact">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="name" type="xs:string"/>
              <xs:element name="surname" type="xs:string"/>
              <xs:element name="telephone" type="xs:string"/>
              <xs:element name="email" type="xs:string"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<addressbook>
  <contact>
    <name>Mary</name>
    <surname>Smith</surname>
    <telephone>1234567890</telephone>
    <email>mary.smith@email.com</email>
  </contact>
  <contact>
    <name>Lucy</name>
    <surname>Baker</surname>
    <telephone>0987654321</telephone>
    <email>lucy.baker@email.com</email>
  </contact>
</addressbook>
```

30